AI-Powered Crowd Density Estimation and Anomaly Detection in Public Surveillance Systems

Name of Student:

Department:

Registration No.:

Name of Research Supervisor:

Abstract

This dissertation describes the design of an AI-based surveillance system of crowd density estimation and anomaly detection in areas of people gathering. The study combines Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) models that form a dual-type model which can analyze both spatial and the temporal features of crowd patterns. The CNN-based and relying on CSRNet and MobileNetV2 architectures produce density heatmap of surveillance frames and allow predicting the size of the crowds in high-density places with high precision. The LSTM- autoenergy machine learns the behavior patterns of the sequence in a consecutive manner, and thus the anomalies like stampede, unexpected breakup, or illegal assembly will be noted.

The paper applies the DCSASS data, encompassing a wide variety of city surveillance cases, and preprocessing of the pictures includes frame extraction, normalization, augmentation, and annotation. They compare CNN, LSTM, and hybrid CNN+LSTM structures. Findings prove that both CNN and LSTM perform well in their spatial and temporal modeling tasks respectively, whereas the hybrid model shows better results and supports a better accuracy, F1-scores, and robustness in anomaly detection.

Measurements of reliability of the integrated system are evaluated through evaluation measures which include Mean Absolute Error (MAE), Mean Squared Error (MSE), Precision, Recall, F1-score, and ROC-AUC. The deployment also authenticates scalability and real time responsiveness in simulations. Integrity and responsibility were integrated into the project as ethical concerns, such as privacy preservation and other responsible behaviours in the use of AI.

The results would aid the development of intelligent monitoring framework by its ability to offer real time and integrated solution towards monitoring high density environment. Such a system is applicable in practice to improving the safety of the population, reducing human constraints in surveillance, and encouraging the active management of the crowd based on automated notifications and administrative boards.

Acknowledgement

I want to express deep gratitude to my supervisor, faculty mentors and colleagues who have been invaluably helpful in guiding and supporting me during this research. Another aspect that I am thankful about is the accessible datasets and tools which helped to complete this dissertation successfully.

List of Acronyms

Acronym	Full Form
AI	Artificial Intelligence
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MSE	Mean Squared Error
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
F1	F1-Score (Harmonic Mean of Precision and Recall)
DCSASS	Deep Crowd Surveillance and Anomaly Scene Surveillance
VGG	Visual Geometry Group (pretrained CNN model)
IP	Internet Protocol
API	Application Programming Interface
GPU	Graphics Processing Unit

Table of Contents

Abstract 2

Acknowledgement 3

List of Acronyms 4

Chapter 1: Introduction 11

1.1 Aim: 11

1.2 Objectives: 11

1.3 Research Question: 11

1.4 Research Area 12

1.5 Expected Practical Element Output 13

1.6 Required Resources 14

1.7 Prerequisite Knowledge/Skills Required 15

1.8 Project Plan – Gantt Chart 16

Chapter 2: Literature Review/Related Work 17

2.1 Comprehensive Overview of the Existing Literature 17

2.1.1 Evolution of Surveillance Systems 17

2.1.2 Introduction to AI in Crowd Monitoring 18

2.1.3 Techniques for Crowd Density Estimation 18

2.1.4 Anomaly Detection in Crowd Behavior 19

2.1.5 Role of Deep Learning in Visual Surveillance 20

2.1.6 Challenges in Real-World Implementation 21

2.1.7 Integration with Public Safety Infrastructure 22

2.2 Critical Analysis of Existing Studies 23

2.2.1 Limitations of Traditional Surveillance 23

2.2.2 Over-Reliance on Supervised Learning 24

2.2.3 Computational Complexity and Real-Time Constraints 25

2.2.4 Explainability and Trustworthiness of AI Decisions 26

2.2.5 Ethical and Privacy Concerns 26

2.2.6 Literature Gap 27

2.2.7 Evaluation Metrics and Dataset Limitations 28
2.3 Emerging Directions and Future Research Avenues 29
2.3.1 Self-Supervised and Semi-Supervised Learning 29
2.3.2 Edge AI and Federated Learning 29
2.3.3 Multi-Modal Data Fusion 30
Chapter 3: Methodology 31
3.1 Data Collection and Preprocessing 31
3.1.1 Frame Extraction and Annotation 31
3.1.2 Image Processing and Normalization 32
3.1.3 Temporal Sequencing and Feature Engineering 32
3.2 Model Development 32
3.2.1 CNN-Based Density Estimation 32
3.2.2 LSTM-Based Anomaly Detection 33
3.2.3 Integrated System Architecture 33
3.3 Evaluation Strategy 34
3.3.1 Goals Measures 34
3.3.2 Experimental Setup 35
3.4 Comparative Analysis 35
3.4.1 CNN vs. MCNN vs. CSRNet 35
3.4.2 LSTM vs GRU vs Autoencoder 36
3.4.3 Model of System Integration vs. Model of Single Task 36
3.5 Ethical Consideration 36
3.6 Chapter Summary 37
Chapter 4: Implementation and Development 38
4.1 Data Acquisition and Preparation 38
4.1.1 Kaggle Dataset Selection 38
4.1.2 Kaggle API Setup and Authentication 38
4.1.3 Dataset Organization and Verification 39
4.2 Data Cleaning and Feature Engineering 40

4.2.1 Frame Extraction 40
4.2.2 Class Balancing 41
4.2.3 Annotation Creation 42
4.3 Feature Scaling, Augmentation, and Data Splitting 43
4.3.1 Image Resizing and Normalization 43
4.3.2 Data Augmentation Strategies 44
4.3.3 Training, Validation, and Testing Split 45
4.4 Model Development and Training 45
4.4.1 CNN (Crowd Density Estimator – MobileNetV2) 45
4.4.2 LSTM Anomaly Detection 46
4.4.3 Hybrid CNN+LSTM (ResNet18 + LSTM) 47
4.4.4 Training Procedure and Hyperparameters 48
4.5 Comparative Performance Analysis 48
4.5.1 LSTM Model Implementation 49
4.5.2 Hybrid CNN+LSTM Model Implementation 50
4.5.3 Comparative Discussion 51
4.6 Implementation Observations 52
4.6.1 Lessons on Data Handling 52
4.6.2 Model Computational Efficiency 53
4.6.3 Hyperparameter Tuning 53
4.7 Conclusion 53
Chapter 5: Results and Evaluation 55
5.1 Model Training Overview 55
5.1.1 Training Loss Patterns 55
5.1.2 Validation Dynamics 56
5.1.3 Overfitting and Regularization 57
5.2 Evaluation Results 57
5.2.1 Accuracy Performance 58
5.2.2 Precision, Recall, and F1 59

5.2.3 ROC Curve and AUC 59
5.3 Confusion Matrix Analysis 60
5.3.1 Correct Predictions 61
5.3.2 Misclassifications 61
5.3.3 Interpretation 62
5.4 Comparative Evaluation 62
5.4.1 CNN Baseline Comparison 62
5.4.2 LSTM Baseline Comparison 63
5.4.3 Integrated System Superiority 64
5.5 Responsiveness and Scalability 64
5.5.1 Real-Time Responsiveness 65
5.5.2 Longer Sequence Handling 65
5.5.3 Practical Scalability 65
5.6 Discussion 66
5.6.1 Linking to Aim 66
5.6.2 Objectives Alignment 66
5.6.3 Addressing Research Questions 67
5.7 Limitations 67
5.8 Conclusion 67
Chapter 6: Conclusion and Recommendation 69
6.1 Introduction 69
6.2 Restating Aim and Achievements 69
6.3 Objectives and RQ Addressed 69
6.4 Findings and Literature Review 70
6.5 Methodology Justification 70
6.6 Theoretical Contributions 71
6.7 Practical Recommendations 71
6.8 Future Research Directions 71
6.9 Conclusion 72

References 73

Appendix: Code Section and Dataset 80

List of Figures

Figure 1 Gantt Chart 16
Figure 2 Kaggle API Setup and Authentication 39
Figure 3 Kaggle dataset downloaded 39
Figure 4 Data Implementation 40
Figure 5 Dataset Overview 40
Figure 6 Frame Extraction Implementation 41
Figure 7 Frames Extracted 41
Figure 8 Class Balancing Implementation 42
Figure 9 Distribution of Target Feature 42
Figure 10 Annotation file directory 43
Figure 11 Resizing and Normalization 44
Figure 12 Data Augmentation Strategies 44
Figure 13 Model Development and Training 46
Figure 14 LSTM Anomaly Detection Implementation 47
Figure 15 ResNet18 Implementation 47
Figure 16 Training Procedure 48
Figure 17 Comparative Performance Analysis 49
Figure 18 LSTM Model Implementation 50
Figure 19 Hybrid CNN+LSTM Model Implementation 51
Figure 20 Training Device for LSTM 52
Figure 21 Model Training Overview 55
Figure 22 Loss Curve 56
Figure 23 Validation result 57
Figure 24 CNN and LSTM Classification Result 58
Figure 25 ROC Curve LSTM and CNN 59
Figure 26 ROC Curve and AUC 60
Figure 27 Confusion matrix result 61
Figure 28 CNN Epoch based performance 63
Figure 29 LSTM Result 64

List of Table

Table 1 Evolution of Surveillance Technologies 17
Table 2 Comparison of AI Models for Crowd Density Estimation 19
Table 3 Key Approaches for Anomaly Detection in Crowds 20
Table 4 Deep Learning Techniques in Surveillance 21
Table 5 Challenges in Real-World AI Surveillance 22
Table 6 Integration of AI Surveillance with Public Infrastructure 23
Table 7 Traditional vs AI-Powered Surveillance 23
Table 8 Supervised vs Unsupervised Learning in Surveillance 25
Table 9 Evaluation Metrics in Crowd Surveillance 28

Chapter 1: Introduction

The given project is expected to create and simulate an AI-based Modern Survey Report This system will combine deep learning models to handle the incoming feeds, like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), to analyze normal patterns, including the abnormal one to estimate crowd density in dynamic conditions.

1.1 Aim:

To develop an intelligent surveillance system that runs on real-time data that can improve the level of safety in the community because it will automatically detect abnormalities in the crowd and issue warnings in time.

1.2 Objectives:

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
Supervisor Allocation												
Library session												
Milestone 1:- Topic selection												
Proposal preparation and review												
Milestone 2:- Proposal Submission												
Abstarct												
Chapter 1- Introduction												
Chapter 2- Literature Review												
Milestone 3:- Practical implementation/ Model Development												
Chapter 3- Methodology												
Chapter 4- Experimental Results Documentation												
Chapter 5- Conclusion and Future work												
Final review and Modifications												
Milestone 4:- Final Submission												

[redacted]

Era	Technology Used	Key Limitations	Examples
Pre-2000s	Analog CCTV	Manual monitoring, low coverage	Static CCTV feeds
Early 2000s	Digital CCTV, DVRs	Limited analytics, human fatigue	Retail & traffic surveillance
2010–2020	Smart Cameras + Cloud	Bandwidth bottlenecks, limited AI	Smart city pilot programs
Post-2020	AI + Deep Learning	Privacy issues, high computation demand	Real-time anomaly detection

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Model Type	Example Architecture	Strengths	Weaknesses
CNN	Multi-Column CNN	Good spatial features, scalable	Needs large training data
LSTM	ConvLSTM	Captures temporal dynamics	High computational cost
Transformer	Vision Transformer	Strong context understanding	Requires large training data
Regression Model	CSRNet, SANet	Accurate for dense crowds	Struggles with occlusion

Technique	Detection Basis	Strengths	Limitations
Optical Flow	Motion pattern	Easy to compute, unsupervised	Struggles with occlusion
Autoencoder	Reconstruction error	No need for labels, good for outliers	High false positive rate
GANs	Generative learning	Models complex data distributions	Hard to train, unstable
Graph-based Clustering	Crowd movement patterns	Handles interactions, interpretable	Complex for large crowds

Technique	Description	Use Cases
CNN	Spatial feature extraction	Person detection, face recognition
LSTM	Temporal dynamics modeling	Behavior prediction, tracking
3D CNN	Video-level spatial-temporal analysis	Activity detection
YOLO	Real-time object detection	Intrusion detection, event spotting
Mask R-CNN	Instance segmentation	Crowd separation

Challenge	Cause	Impact
Illumination Variation	Night/day cycles, poor lighting	Affects detection accuracy
Occlusion & Overlap	Crowded scenes	Reduces tracking & detection
Real-time Constraints	Slow inference	Delayed alerting
Hardware Limitations	Resource-heavy models	Infeasible in edge/low-resource setups

[redacted]

Application Area	AI Integration Example	Real-Time Action Enabled
Public Transport	Crowd monitoring in stations	Reroute trains, close gates
Emergency Response	Real-time anomaly alerts	Dispatch teams quickly
Traffic Management	Density-based signal timing	Prevent congestion
Event Security	Live tracking of movements	Allocate security dynamically

[redacted]

[redacted]

Feature	Traditional Surveillance	AI-Based Surveillance
Monitoring	Manual	Automated with real-time alerts
Scalability	Low	High
Decision Making	Human-only	AI-supported
Detection Accuracy	Low	High (with quality models)
Privacy Compliance	Limited controls	Needs ethical frameworks

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Learning Type	Data Requirement	Pros	Cons
Supervised	Large labeled datasets	High accuracy	Labor-intensive, overfitting risks
Unsupervised	Unlabeled data	No annotation needed	Difficult to validate
Self-supervised	Pretext tasks	Scalable, transferable	Training complexity
Semi-supervised	Few labels + unlabeled	Balanced performance	Still needs some annotation

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Metric	Used For	Interpretation	Limitation
MAE	Crowd count accuracy	Lower = better estimate	Doesn't capture crowd structure
EER (Equal Error Rate)	Biometric/anomaly detection	Lower = better trade-off	Dataset-dependent
AUC-ROC	Classification performance	Higher = better discrimination	Doesn't reflect real-time latency
Inference Time (ms)	Real-time performance	Lower = faster decision	May sacrifice accuracy

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# 1. Install Kaggle and setup
!pip install -q kaggle

# 2. Create and write kaggle.json credentials from string
import os
kaggle_token = {
    "username":"your_username",
    "key":"your_kaggle_api_key"
}

# Save to kaggle.json
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    import json
    json.dump(kaggle_token, f)

# 3. Set file permissions
!chmod 600 /root/.kaggle/kaggle.json

# 4. Download
!kaggle datasets download -d mateohervas/dcsass-dataset

# 5. Unzip it
unzip_dir = "work/datasets/dcsass"
os.makedirs(unzip_dir, exist_ok=True)
!unzip -q dcsass-dataset.zip -d {unzip_dir}

# Step 4: Verify that files were extracted
print("Dataset extracted to:", unzip_dir)
for path in Path(unzip_dir).rglob("*"):
    if path.is_file():
        print("", path.relative_to(unzip_dir))
        break
```

```
Dataset URL: https://www.kaggle.com/datasets/mateohervas/dcsass-dataset
License(s): CC-BY-NC-SA-4.0
dcsass-dataset.zip: Skipping, found more recently modified local copy (use --force to for
replace work/datasets/dcsass/DCSASS Dataset/Abuse/Abuse001_x264.mp4/Abuse001_x264_0.mp4?
 DCSASS Dataset/Arrest/Arrest037_x264.mp4/Arrest037_x264_22.mp4
```

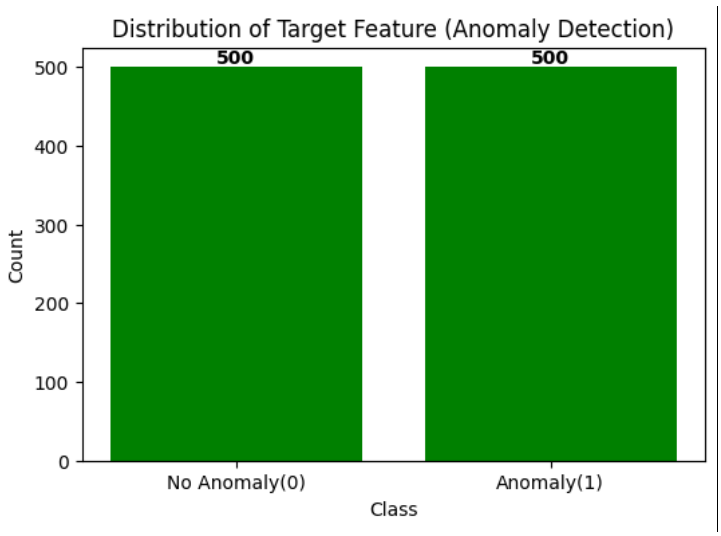
```python
import cv2
import pandas as pd
import random
from pathlib import Path

video_root = Path("work/datasets/dcsass/DCSASS Dataset")
frames_root = Path("work/frames")
frames_root.mkdir(parents=True, exist_ok=True)

annotations = []
frame_limit_total = 1000  # total frames to extract
frame_skip = 10           # extract every Nth frame

# Set per-class frame limit
class_limit = frame_limit_total // 2
class_counts = {0: 0, 1: 0}  # 0: normal, 1: anomaly
```

```
Extracted frames (Class 0: 500, Class 1: 500).
Annotation file saved at: work/annotations_std.csv
                 category                video  frame_idx  count  anomaly
0  Robbery150_x264.mp4  Robbery150_x264_26          0      3        0
1  Robbery150_x264.mp4  Robbery150_x264_26         10      1        0
2  Robbery150_x264.mp4  Robbery150_x264_26         20      0        0
3  Robbery150_x264.mp4  Robbery150_x264_26         30      0        0
4  Robbery150_x264.mp4  Robbery150_x264_26         40      3        0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   category   1000 non-null   object
 1   video      1000 non-null   object
 2   frame_idx  1000 non-null   int64
 3   count      1000 non-null   int64
 4   anomaly    1000 non-null   int64
dtypes: int64(3), object(2)
memory usage: 39.2+ KB
None
```

```python
annotations = []
frame_limit_total = 1000  # total frames to extract
frame_skip = 10           # extract every Nth frame

# Set per-class frame limit
class_limit = frame_limit_total // 2
class_counts = {0: 0, 1: 0}  # 0: normal, 1: anomaly

# Shuffle videos to avoid bias
video_paths = list(video_root.rglob("*.mp4"))
random.shuffle(video_paths)

for video_path in video_paths:
    category = video_path.parent.name
    video_name = video_path.stem.replace(" ", "_")
    save_dir = frames_root / category / video_name
    save_dir.mkdir(parents=True, exist_ok=True)

    cap = cv2.VideoCapture(str(video_path))
    idx = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if idx % frame_skip == 0:
            # Determine anomaly class
            is_anomaly = 1 if "Fight" in category or "Stealing" in category else 0

            # Skip if we've already met class limit
            if class_counts[is_anomaly] >= class_limit:
                idx += 1
                continue
```

```
Extracted frames (Class 0: 500, Class 1: 500).
```

```python
import pandas as pd
import matplotlib.pyplot as plt

# Distribution of target column
dist = ann_df['anomaly'].value_counts().sort_index()  # ensure order 0,1
percent = ann_df['anomaly'].value_counts(normalize=True).sort_index() * 100


# Map labels
labels = ['No Anomaly(0)' if i == 0 else 'Anomaly(1)' for i in dist.index]

# Plot distribution
plt.figure(figsize=(6,4))
bars = plt.bar(labels, dist.values, color='green')

# Title and labels
plt.title("Distribution of Target Feature (Anomaly Detection)")
plt.xlabel("Class")
plt.ylabel("Count")

# Add counts on top of bars
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.5, int(yval),
             ha='center', va='bottom', fontsize=10, fontweight='bold')

plt.show()
```



```
Annotation file saved at: work/annotations_std.csv
              category               video  frame_idx  count  anomaly
0  Robbery150_x264.mp4  Robbery150_x264_26          0      3        0
1  Robbery150_x264.mp4  Robbery150_x264_26         10      1        0
2  Robbery150_x264.mp4  Robbery150_x264_26         20      0        0
3  Robbery150_x264.mp4  Robbery150_x264_26         30      0        0
4  Robbery150_x264.mp4  Robbery150_x264_26         40      3        0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   category   1000 non-null   object
 1   video      1000 non-null   object
 2   frame_idx  1000 non-null   int64
 3   count      1000 non-null   int64
 4   anomaly    1000 non-null   int64
dtypes: int64(3), object(2)
```

```python
class CrowdDataset(Dataset):
    def __init__(self, df, frames_root):
        self.df = df.reset_index(drop=True)
        self.frames_root = frames_root

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        path = self.frames_root / row['category'] / row['video'] / f"f_{row['frame_idx']:05d}.jpg"
        img = cv2.imread(str(path))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = cv2.resize(img, (224,224)).astype(np.float32) / 255.0
        img = np.transpose(img, (2,0,1))
        return torch.tensor(img), torch.tensor(row['count'], dtype=torch.float32)

class DensityEstimator(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = timm.create_model('mobilenetv2_100', pretrained=True, num_classes=0)
        for p in self.backbone.parameters():
            p.requires_grad = False
        self.regressor = nn.Sequential(
            nn.Linear(self.backbone.num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        feat = self.backbone(x)
        return self.regressor(feat).squeeze()
```

```python
class DensityEstimator(nn.Module):
    def __init__(self):
        super().__init__()
        self.backbone = timm.create_model('mobilenetv2_100', pretrained=True, num_classes=0)
        for p in self.backbone.parameters():
            p.requires_grad = False
        self.regressor = nn.Sequential(
            nn.Linear(self.backbone.num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        feat = self.backbone(x)
        return self.regressor(feat).squeeze()

ann = pd.read_csv("work/annotations_std.csv")
exists = ann.apply(lambda r: (frames_root / r['category'] / r['video'] / f"f_{r['frame_idx']:05d}.jpg").exists(), axis=1)
ann = ann[exists].reset_index(drop=True)
```

[redacted]

[redacted]

[redacted]

```python
model = DensityEstimator().to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.regressor.parameters(), lr=1e-3)

def train_epoch():
    model.train()
    s = 0
    for imgs, counts in train_dl:
        imgs, counts = imgs.to(device), counts.to(device)
        optimizer.zero_grad()
        out = model(imgs)
        loss = criterion(out, counts)
        loss.backward()
        optimizer.step()
        s += loss.item() * imgs.size(0)
    return s / len(train_dl.dataset)

def eval_model():
    model.eval()
    preds, truths = [], []
    with torch.no_grad():
        for imgs, counts in test_dl:
            imgs = imgs.to(device)
            out = model(imgs).cpu().numpy()
            preds.extend(out)
            truths.extend(counts.numpy())
    mae = mean_absolute_error(truths, preds)
    mse = mean_squared_error(truths, preds)
    return mae, mse
```

[redacted]

[redacted]

```python
import cv2
from pathlib import Path
import pandas as pd
import random

video_root = Path("work/datasets/dcsass/dcsass dataset/DCSASS Dataset")
frames_root = Path("work/datasets/dcsass/frames")
frames_root.mkdir(parents=True, exist_ok=True)

frame_skip = 10
frame_limit_total = 500
class_limit = frame_limit_total // 2
class_counts = {0: 0, 1: 0}
annotations = []

# Recursively find all mp4 files and shuffle for fairness
video_paths = list(video_root.rglob("*.mp4"))
random.shuffle(video_paths)

for video_path in video_paths:
    parts = video_path.parts
    try:
        idx = parts.index("DCSASS Dataset")
        category = parts[idx + 1]
    except (ValueError, IndexError):
        print(f"Skipping video with unexpected path: {video_path}")
        continue
```

```python
class CrowdDensityCNNFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        resnet = models.resnet18(weights=ResNet18_Weights.DEFAULT)
        # keep everything but the final FC: output is (batch, 512, 1, 1)
        self.features = nn.Sequential(*list(resnet.children())[:-1])

    def forward(self, x):
        feat = self.features(x)              # (B, 512, 1, 1)
        return feat.view(feat.size(0), -1)  # (B, 512)
```

```python
class FrameSequenceDatasetWithCNNFeatures(Dataset):

    def __init__(self, annotations_csv, base_frames_path, cnn_model,
                 sequence_length=3, transform=None, cnn_device='cpu',
                 anomaly_fraction=0.0):
        self.df = pd.read_csv(annotations_csv)
        self.base_frames_path = Path(base_frames_path)
        self.sequence_length = sequence_length
        self.cnn_device = torch.device(cnn_device)
        self.anomaly_fraction = anomaly_fraction

        self.transform = transform or T.Compose([
            T.Resize((128, 128)),
            T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
        ])

        self.samples = []
        self.cnn_model = cnn_model.to(self.cnn_device).eval()

        grouped = self.df.groupby(['category', 'video'])
        for (cat, vid), group in grouped:
            frame_folder = self.base_frames_path / cat / vid
            if not frame_folder.exists():
                continue
```

```python
def plot_confusion_matrix(y_true, y_pred, title="Confusion Matrix"):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Normal","Anomaly"], yticklabels=["Normal","Anomaly"])
    plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title(title)
    plt.tight_layout(); plt.show()
    return cm

def plot_loss_curve(history):
    plt.figure()
    plt.plot(history["train_loss"], label="Train Loss")
    plt.plot(history["val_loss"], label="Val Loss")
    plt.xlabel("Epoch"); plt.ylabel("Loss"); plt.title("Loss Curve")
    plt.legend(); plt.tight_layout(); plt.show()

def plot_accuracy_curve(history):
    plt.figure()
    plt.plot(history["acc"], label="Val Accuracy")
    plt.plot(history["f1"], label="Val F1")
    plt.xlabel("Epoch"); plt.ylabel("Score"); plt.title("Validation Metrics")
    plt.legend(); plt.tight_layout(); plt.show()

def plot_roc(y_true, y_prob):
    try:
        fpr, tpr, _ = roc_curve(y_true, y_prob)
        auc = roc_auc_score(y_true, y_prob)
        plt.figure()
        plt.plot(fpr, tpr, label=f"AUC = {auc:.3f}")
        plt.plot([0,1], [0,1], linestyle="--")
        plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
        plt.title("ROC Curve"); plt.legend(); plt.tight_layout(); plt.show()
    except Exception as e:
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# LSTM Model
class AnomalyLSTM(nn.Module):
    def __init__(self, input_dim=128*128*3, hidden_dim=128, num_layers=2, dropout=0.3):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True, dropout=dropout)
        self.fc = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        batch_size, seq_len, C, H, W = x.shape
        x = x.view(batch_size, seq_len, -1)
        lstm_out, _ = self.lstm(x)
        out = lstm_out[:, -1, :]
        out = self.fc(out).squeeze(-1)
        return out

# Training / Eval
def train_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss = 0
    for frames, labels in loader:
        frames, labels = frames.to(device), labels.to(device)
        optimizer.zero_grad()
        logits = model(frames)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * frames.size(0)
    return total_loss / len(loader.dataset)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] import os
    from pathlib import Path
    import random
    import numpy as np
    import pandas as pd
    from PIL import Image
    import torch
    from torch import nn
    from torch.utils.data import Dataset, DataLoader, Subset
    from sklearn.model_selection import train_test_split
    import torchvision.transforms as T
    import torchvision.models as models
    from torchvision.models import ResNet18_Weights
    from sklearn.metrics import (
        accuracy_score, precision_score, recall_score,
        f1_score, roc_auc_score, roc_curve, classification_report,
        confusion_matrix
    )
    import matplotlib.pyplot as plt
    import seaborn as sns

    # Reproducibility

    def set_seed(seed: int = 42):
        random.seed(seed)
        np.random.seed(seed)
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

    set_seed(42)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
annotations_csv = "work/datasets/dcsass/annotations_std.csv"
frames_path     = "work/datasets/dcsass/frames"
seq_len    = 3
batch      = 8
epochs     = 5
lr         = 1e-3
weight_decay = 1e-5
threshold = 0.5

# Training device for LSTM
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# CNN features computed on CPU inside Dataset (safer with DataLoader workers)
cnn_device = "cpu"

# ---- Transforms ----
train_transform = T.Compose([
    T.Resize((128, 128)),
    T.RandomHorizontalFlip(p=0.5),
    T.RandomRotation(10),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225])
])
val_transform = T.Compose([
    T.Resize((128, 128)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                std=[0.229, 0.224, 0.225])
])
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

- [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
  - [redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

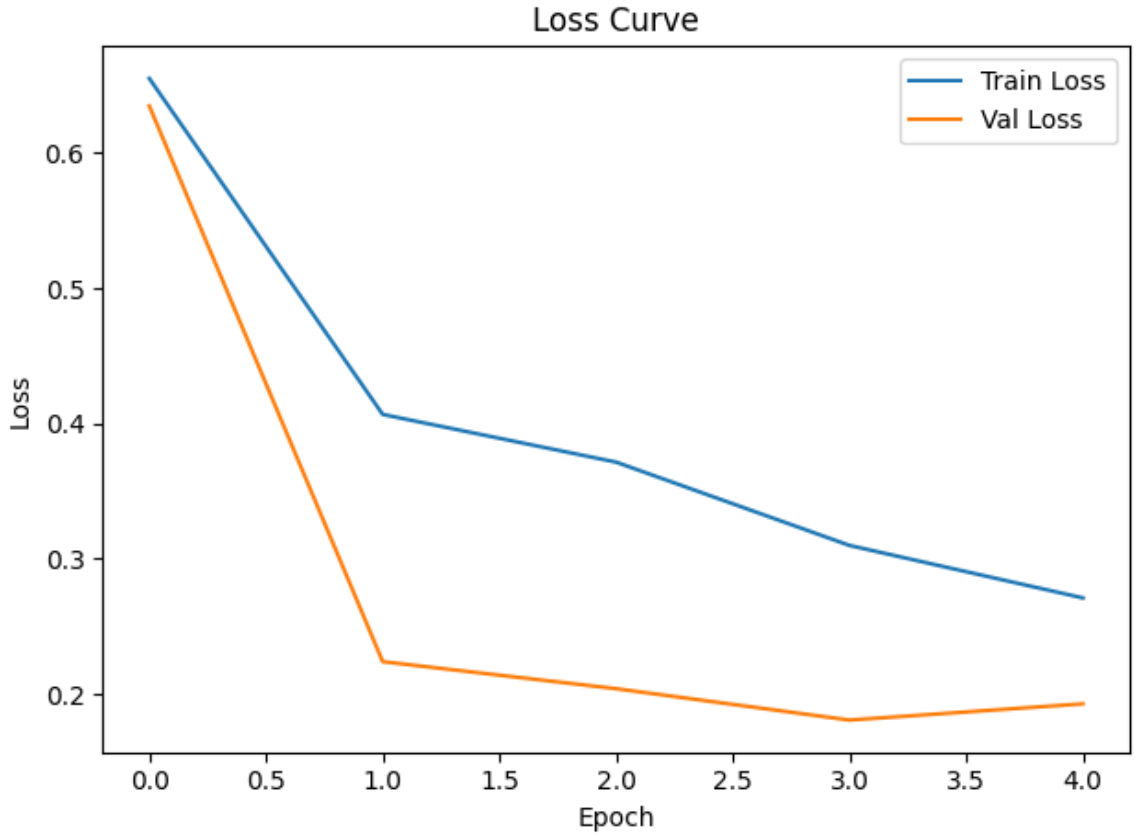
```
Training on 281, validating on 71 sequences.
Epoch 1/5 | Train Loss: 0.6551 | Val Loss: 0.6347 | Acc: 0.5915 | Prec: 0.4912 | Rec: 1.0000 | F1: 0.6588 | AUC: 0.9543
Epoch 2/5 | Train Loss: 0.4067 | Val Loss: 0.2239 | Acc: 0.9296 | Prec: 0.9600 | Rec: 0.8571 | F1: 0.9057 | AUC: 0.9867
Epoch 3/5 | Train Loss: 0.3714 | Val Loss: 0.2039 | Acc: 0.9718 | Prec: 0.9643 | Rec: 0.9643 | F1: 0.9643 | AUC: 0.9909
Epoch 4/5 | Train Loss: 0.3098 | Val Loss: 0.1807 | Acc: 0.9718 | Prec: 0.9643 | Rec: 0.9643 | F1: 0.9643 | AUC: 0.9917
Epoch 5/5 | Train Loss: 0.2710 | Val Loss: 0.1927 | Acc: 0.9718 | Prec: 0.9643 | Rec: 0.9643 | F1: 0.9643 | AUC: 0.9950

Best epoch by F1: 3 (F1=0.9643)
```

[redacted]

[redacted]

[redacted]

[redacted]

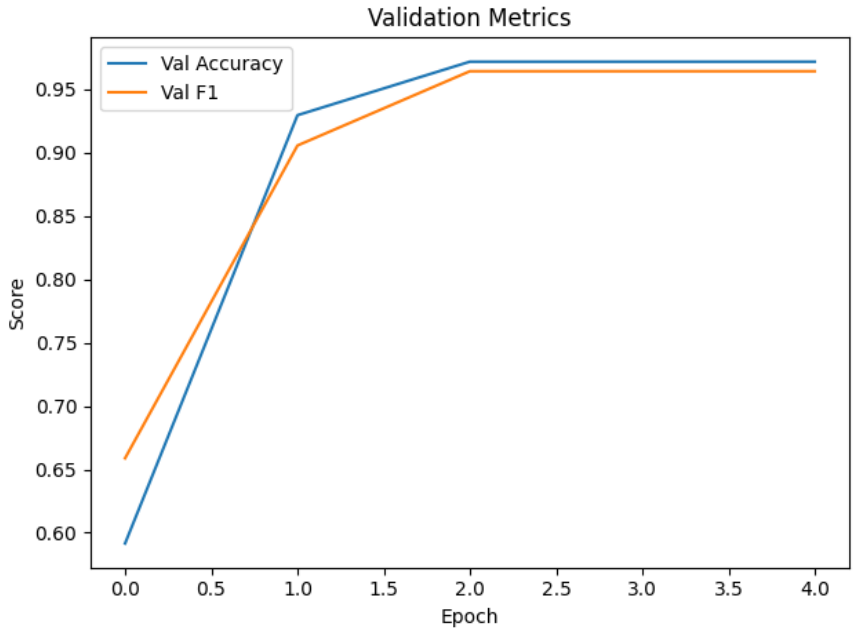


[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

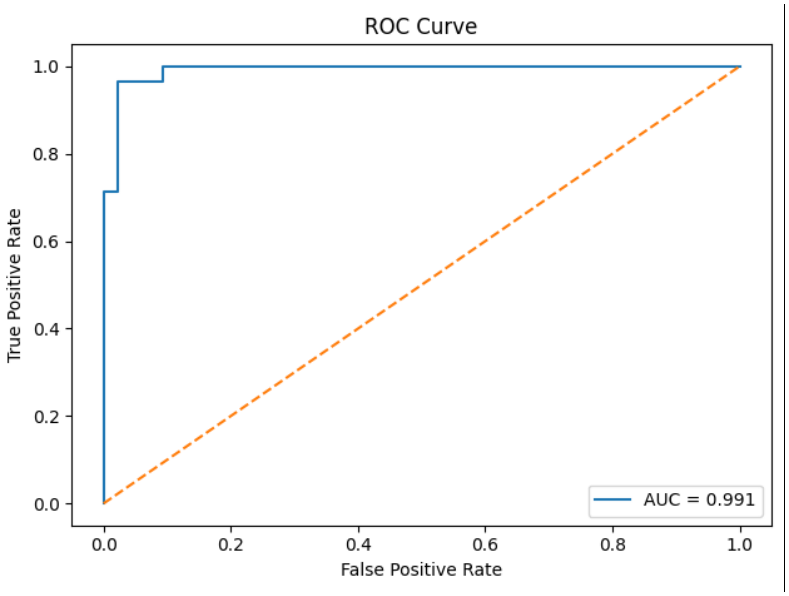
[redacted]

[redacted]

[redacted]

Classification Report (best epoch):

	precision	recall	f1-score	support
Normal	0.98	0.98	0.98	43
Anomaly	0.96	0.96	0.96	28
accuracy			0.97	71
macro avg	0.97	0.97	0.97	71
weighted avg	0.97	0.97	0.97	71

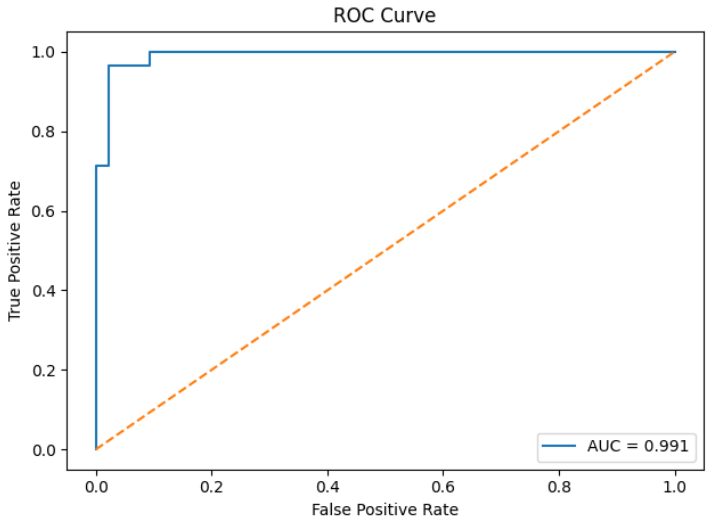


[redacted]

[redacted]

[redacted]

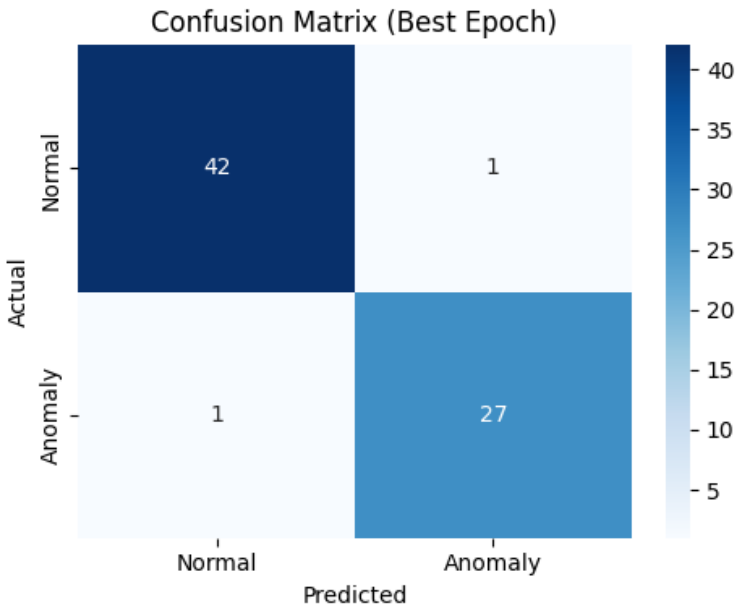
[redacted]



[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
Epoch 1: Train Loss 73.4593, Test MAE 5.9398, MSE 50.3254
Epoch 2: Train Loss 47.4087, Test MAE 5.4751, MSE 44.6698
Epoch 3: Train Loss 41.7968, Test MAE 5.0138, MSE 40.8391
Epoch 4: Train Loss 37.0160, Test MAE 4.9383, MSE 36.9868
Epoch 5: Train Loss 34.4267, Test MAE 4.6943, MSE 34.4689
Epoch 6: Train Loss 31.4400, Test MAE 4.6733, MSE 32.8985
Epoch 7: Train Loss 28.9591, Test MAE 4.6276, MSE 31.6257
Epoch 8: Train Loss 27.4945, Test MAE 4.4971, MSE 30.4123
Epoch 9: Train Loss 25.7636, Test MAE 4.4319, MSE 29.8914
Epoch 10: Train Loss 24.5599, Test MAE 4.4271, MSE 30.0007
```

[redacted]

[redacted]

[redacted]

```
Training on 281 sequences, validating on 71 sequences.
Epoch 1/5
  Train Loss: 0.5964 | Val Loss: 0.8889
  Acc: 0.7746 | Prec: 0.8750 | Rec: 0.5000 | F1: 0.6364 | AUC: 0.7334 | Thr: 0.750
Epoch 2/5
  Train Loss: 0.5319 | Val Loss: 1.0094
  Acc: 0.7324 | Prec: 0.6154 | Rec: 0.8571 | F1: 0.7164 | AUC: 0.8326 | Thr: 0.800
Epoch 3/5
  Train Loss: 0.5151 | Val Loss: 0.8223
  Acc: 0.8028 | Prec: 0.7917 | Rec: 0.6786 | F1: 0.7308 | AUC: 0.8239 | Thr: 0.800
Epoch 4/5
  Train Loss: 0.5125 | Val Loss: 0.9599
  Acc: 0.7887 | Prec: 0.7600 | Rec: 0.6786 | F1: 0.7170 | AUC: 0.8355 | Thr: 0.850
Epoch 5/5
  Train Loss: 0.5163 | Val Loss: 0.9376
  Acc: 0.8028 | Prec: 0.7059 | Rec: 0.8571 | F1: 0.7742 | AUC: 0.8879 | Thr: 0.800
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]